

Table of Contents

1. [1. Complexity Cheat Sheet](#)
2. [2. Arrays & Two Pointers](#)
3. [3. Sliding Window](#)
4. [4. Prefix Sum & Kadane's](#)
5. [5. Sorting Algorithms](#)
6. [6. Binary Search Patterns](#)
7. [7. Recursion & Backtracking](#)
8. [8. Linked List Patterns](#)
9. [9. Stack Patterns](#)
10. [10. Queue & Deque Patterns](#)
11. [11. Trees](#)
12. [12. Binary Search Trees](#)
13. [13. Heaps / Priority Queue](#)
14. [14. Tries](#)
15. [15. Graphs](#)
16. [16. Dynamic Programming](#)
17. [17. Greedy Algorithms](#)
18. [18. Bit Manipulation](#)
19. [19. Math for DSA](#)
20. [20. How to Practice \(Roadmap\)](#)

The Complete DSA Handbook (C++) — Placement Preparation

Every major pattern, from scratch, with intuition, C++ code, and time/space complexity. Read top to bottom once, then use as a lookup reference while solving problems.

1. Complexity Cheat Sheet

Notation	Name	Example
$O(1)$	Constant	array index access
$O(\log n)$	Logarithmic	binary search
$O(n)$	Linear	single loop
$O(n \log n)$	Linearithmic	merge sort, sort()
$O(n^2)$	Quadratic	nested loops
$O(2^n)$	Exponential	subsets, brute recursion
$O(n!)$	Factorial	permutations

Rule of thumb for constraints (competitive/placement):

- $n \leq 10 \rightarrow O(n!)$ or $O(2^n \cdot n)$ fine
- $n \leq 20 \rightarrow O(2^n)$
- $n \leq 500 \rightarrow O(n^3)$
- $n \leq 5,000 \rightarrow O(n^2)$
- $n \leq 10^6 \rightarrow O(n \log n)$
- $n \leq 10^8 \rightarrow O(n)$ (tight)

Space complexity = extra memory used, *not counting input*, as a function of n (auxiliary arrays, recursion stack depth, hash maps, etc.)

2. Arrays & Two Pointers

Pattern: Two Pointers (opposite ends)

What it is: Two variables ("pointers") start at opposite ends of an array and move toward each other, each step narrowing down the search — instead of checking every pair with nested loops.

Intuition: When array is sorted (or can be sorted) and you're looking for a pair/triplet satisfying a sum condition, moving two pointers from both ends avoids the $O(n^2)$ brute force.

Use when: "pair with target sum", "container with most water", "remove duplicates", "3Sum".

Problem: Given a sorted array and a target value, find the indices of two numbers that add up to the target.

Example: `nums = [2,7,11,15]`, `target = 9` → Output: `[0,1]` (`nums[0]+nums[1] = 2+7 = 9`)

```
// Two Sum on SORTED array – return indices (1-indexed) or values
#include <bits/stdc++.h>
using namespace std;
```

```

pair<int,int> twoSumSorted(vector<int>& a, int target) {
    int l = 0, r = a.size() - 1;
    while (l < r) {
        int sum = a[l] + a[r];
        if (sum == target) return {l, r};
        else if (sum < target) l++;    // need bigger sum
        else r--;                    // need smaller sum
    }
    return {-1, -1};
}

```

Time: $O(n)$ — each pointer moves at most n times total. **Space:** $O(1)$.

Practice: [Two Sum II - Input Array Is Sorted](#) · [3Sum](#) · [Container With Most Water](#)

Pattern: Fast & Slow Pointers (same direction — in-place compaction)

What it is: One pointer ("slow") marks where the next valid element should go, while another pointer ("fast") scans ahead looking for it — lets you rewrite an array in place without extra memory.

Recognize it by: "remove duplicates in-place", "move zeroes to the end", "in-place" + " $O(1)$ extra space" on an array.

Problem: Given a sorted array, remove the duplicates in-place so each element appears only once, and return the new length.

Example: `nums = [1,1,2,2,3]` → Output: length = 3, array becomes `[1,2,3,_,_]_`

```

// Remove duplicates from sorted array in-place
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int slow = 0;
    for (int fast = 1; fast < a.size(); fast++) {
        if (a[fast] != a[slow]) {
            slow++;
            a[slow] = a[fast];
        }
    }
    return slow + 1; // new length
}

```

Time: $O(n)$. **Space:** $O(1)$.

Practice: [Remove Duplicates from Sorted Array](#) · [Move Zeroes](#)

Pattern: Dutch National Flag (3-way partition)

What it is: A specialised 3-pointer technique (named after the Dutch flag's three colour bands) for sorting an array that only contains 3 distinct values, in a single left-to-right pass.

Recognize it by: the array contains only a small fixed set of distinct values (like 0/1/2) and you must sort it in one pass without extra space.

Intuition: Sort an array of 0s, 1s, 2s in one pass using low/mid/high pointers.

Problem: Given an array containing only the values 0, 1, and 2, sort it in-place in a single pass.

Example: `nums = [2,0,2,1,1,0]` → Output: `[0,0,1,1,2,2]`

```

void sortColors(vector<int>& a) {
    int low = 0, mid = 0, high = a.size() - 1;
    while (mid <= high) {
        if (a[mid] == 0) swap(a[low++], a[mid++]);
    }
}

```

```

        else if (a[mid] == 1) mid++;
        else swap(a[mid], a[high--]); // don't increment mid here
    }
}

```

Time: $O(n)$. **Space:** $O(1)$.

Practice: [Sort Colors](#)

3. Sliding Window

Intuition: For contiguous subarray/substring problems, instead of recomputing sum/state for every window ($O(n^2)$), expand the right edge and shrink the left edge, maintaining a running state.

3a. Fixed-size window

What it is: A "window" of exactly k elements slides across the array one step at a time; instead of resumming all k elements every step, you just add the new element and drop the old one.

Recognize it by: "subarray of size k ", "every k consecutive elements", "window of fixed length k ".

Problem: Given an array and an integer k , find the maximum sum among all contiguous subarrays of size exactly k .

Example: $a = [2, 1, 5, 1, 3, 2]$, $k = 3 \rightarrow$ Output: 9 (subarray $[5, 1, 3]$)

Dry Run:

```

window [0..2] = [2, 1, 5] -> sum = 8, best = 8
slide: +a[3](1) -a[0](2) -> sum = 7, best = 8
slide: +a[4](3) -a[1](1) -> sum = 9, best = 9
slide: +a[5](2) -a[2](5) -> sum = 6, best = 9
Answer: 9

```

```

// Max sum of subarray of size k
int maxSumFixedWindow(vector<int>& a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    int best = sum;
    for (int i = k; i < a.size(); i++) {
        sum += a[i] - a[i - k]; // slide window by 1
        best = max(best, sum);
    }
    return best;
}

```

Time: $O(n)$. **Space:** $O(1)$.

Practice: [Maximum Average Subarray I](#) · [Sliding Window Maximum](#)

3b. Variable-size window (shrinkable)

What it is: The window's right edge keeps expanding to include more elements, and whenever some condition is satisfied, the left edge shrinks inward — the window's size adapts to the data instead of staying fixed.

Recognize it by: "smallest/shortest subarray such that...", "longest subarray with at most/at least...", a running sum or count that should stay within a limit.

Problem: Given an array of positive integers and a target sum, find the length of the shortest contiguous subarray whose sum is $\geq$ target.

Example: target = 7, a = [2,3,1,2,4,3] → Output: 2 (subarray [4,3])

Dry Run:

```
right=0 a[0]=2 -> sum=2
right=1 a[1]=3 -> sum=5
right=2 a[2]=1 -> sum=6
right=3 a[3]=2 -> sum=8
    window[0..3] len=4 >= target -> best=4
    shrink: sum -= a[0](2) -> left=1
right=4 a[4]=4 -> sum=10
    window[1..4] len=4 >= target -> best=4
    shrink: sum -= a[1](3) -> left=2
    window[2..4] len=3 >= target -> best=3
    shrink: sum -= a[2](1) -> left=3
right=5 a[5]=3 -> sum=9
    window[3..5] len=3 >= target -> best=3
    shrink: sum -= a[3](2) -> left=4
    window[4..5] len=2 >= target -> best=2
    shrink: sum -= a[4](4) -> left=5
Answer: 2
```

```
// Smallest subarray with sum >= target
int minSubArrayLen(int target, vector<int>& a) {
    int n = a.size(), left = 0, sum = 0, best = INT_MAX;
    for (int right = 0; right < n; right++) {
        sum += a[right];
        while (sum >= target) { // shrink while condition holds
            best = min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    return best == INT_MAX ? 0 : best;
}
```

Time: O(n) — left pointer moves forward at most n times total. **Space:** O(1).

Practice: [Minimum Size Subarray Sum](#) · [Longest Repeating Character Replacement](#)

3c. Window with hashmap (longest substring without repeating chars)

What it is: Same sliding-window idea, but a hashmap tracks extra information (like the last-seen position of each character) so the window can jump forward correctly instead of shrinking one step at a time.

Recognize it by: "longest substring with at most k distinct characters", "without repeating characters", "contains all characters of..." — window + character frequency.

Problem: Given a string, find the length of the longest substring that doesn't contain any repeated characters.

Example: "abcabcbb" → Output: 3 (the substring "abc")

Dry Run:

```
window='a' len=1 best=1
window='ab' len=2 best=2
window='abc' len=3 best=3
right=3 c='a' seen before at 0 (>= left) -> left jumps to 1
    window='bca' len=3 best=3
right=4 c='b' seen before at 1 (>= left) -> left jumps to 2
    window='cab' len=3 best=3
right=5 c='c' seen before at 2 (>= left) -> left jumps to 3
    window='abc' len=3 best=3
right=6 c='b' seen before at 4 (>= left) -> left jumps to 5
    window='cb' len=2 best=3
```

```
right=7 c='b' seen before at 6 (>= left) -> left jumps to 7
window='b' len=1 best=3
Answer: 3
```

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char,int> lastSeen;
    int best = 0, left = 0;
    for (int right = 0; right < s.size(); right++) {
        char c = s[right];
        if (lastSeen.count(c) && lastSeen[c] >= left)
            left = lastSeen[c] + 1; // jump left past the duplicate
        lastSeen[c] = right;
        best = max(best, right - left + 1);
    }
    return best;
}
```

Time: $O(n)$. **Space:** $O(\min(n, \text{alphabet size}))$.

Practice: [Longest Substring Without Repeating Characters](#) · [Permutation in String](#) · [Minimum Window Substring](#)

4. Prefix Sum & Kadane's

Prefix Sum

What it is: Precompute a running total as you go through the array once, so that the sum of ANY range $[l, r]$ can later be answered instantly with a subtraction, instead of re-adding elements every query.

Recognize it by: many repeated "sum of range $[l, r]$ " queries on the same array, or "subarray sum equals K ".

Intuition: Precompute cumulative sums so any range sum query is $O(1)$.

Problem: Given an array, answer multiple queries that each ask for the sum of elements between index l and r (inclusive), as fast as possible per query.

Example: $a = [1, 2, 3, 4, 5]$, query(1,3) $\rightarrow$ Output: 9 (2+3+4)

```
vector<long long> buildPrefix(vector<int>& a) {
    vector<long long> pre(a.size() + 1, 0);
    for (int i = 0; i < a.size(); i++) pre[i+1] = pre[i] + a[i];
    return pre;
}
// sum of range [l, r] inclusive = pre[r+1] - pre[l]
```

Time: $O(n)$ build, $O(1)$ query. **Space:** $O(n)$.

Practice: [Range Sum Query - Immutable](#) · [Subarray Sum Equals K](#)

Variant — subarray sum equals K (using hashmap of prefix sums):

```
int subarraySum(vector<int>& a, int k) {
    unordered_map<long long,int> freq;
    freq[0] = 1;
    long long sum = 0;
    int count = 0;
    for (int x : a) {
        sum += x;
        if (freq.count(sum - k)) count += freq[sum - k];
        freq[sum]++;
    }
    return count;
}
```

}

Time: $O(n)$. **Space:** $O(n)$.

Kadane's Algorithm (Maximum Subarray Sum)

What it is: A single left-to-right pass where, at each element, you decide: "is it better to keep extending my current run, or abandon it and start fresh from here?" — this greedy local decision gives the global best answer.

Recognize it by: "maximum sum subarray", "maximum product subarray", "best contiguous run" in a single array.

Intuition: At each index, either extend the previous subarray or start fresh — whichever gives a bigger sum.

Problem: Given an integer array (which may contain negative numbers), find the contiguous subarray with the largest sum.

Example: $a = [-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow$ Output: 6 (subarray $[4, -1, 2, 1]$)

```

int maxSubArray(vector<int>& a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.size(); i++) {
        cur = max(a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}

```

Time: $O(n)$. **Space:** $O(1)$.**Practice:** [Maximum Subarray](#) · [Maximum Product Subarray](#)

5. Sorting Algorithms

Algorithm	Time (avg)	Time (worst)	Space	Stable?
Bubble/Insertion/Selection	$O(n^2)$	$O(n^2)$	$O(1)$	Bubble/Insertion: yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(k)$	Yes

Merge Sort

What it is: A "divide and conquer" sort: keep splitting the array in half until pieces have 1 element (already sorted), then merge pairs of sorted pieces back together, always keeping the result sorted.

Recognize it by: you need a *stable* $O(n \log n)$ sort, or the merge step itself is reused to solve another problem (like counting inversions).

Intuition: Divide array into halves, sort each half, merge two sorted halves in linear time.

Problem: Sort an array of integers into ascending order.

Example: $a = [5, 2, 4, 1, 3] \rightarrow$ Output: $[1, 2, 3, 4, 5]$

```

void merge(vector<int>& a, int l, int m, int r) {

```

```

vector<int> left(a.begin()+l, a.begin()+m+1);
vector<int> right(a.begin()+m+1, a.begin()+r+1);
int i = 0, j = 0, k = l;
while (i < left.size() && j < right.size())
    a[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
while (i < left.size()) a[k++] = left[i++];
while (j < right.size()) a[k++] = right[j++];
}
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m+1, r);
    merge(a, l, m, r);
}

```

Time: $O(n \log n)$ always. **Space:** $O(n)$ for merging.

Practice: [Sort an Array](#) · [Merge Sorted Array](#)

Quick Sort (Lomuto partition)

What it is: Pick one element (the "pivot"), rearrange the array so everything smaller ends up on its left and everything bigger on its right, then repeat the same process independently on each side.

Recognize it by: you need in-place $O(n \log n)$ average sort, or the partition step is reused (e.g. Kth largest via quickselect).

Intuition: Pick a pivot, partition array so smaller elements go left, larger go right, recurse on both sides.

Problem: Sort an array of integers into ascending order, in-place, using a pivot-partitioning strategy.

Example: $a = [5, 2, 4, 1, 3] \rightarrow \text{Output: } [1, 2, 3, 4, 5]$

```

int partition(vector<int>& a, int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] < pivot) swap(a[++i], a[j]);
    swap(a[i+1], a[high]);
    return i + 1;
}
void quickSort(vector<int>& a, int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

```

Time: $O(n \log n)$ average, $O(n^2)$ worst (already sorted + bad pivot). **Space:** $O(\log n)$ recursion stack.

Practice: [Sort an Array](#) · [Kth Largest Element in an Array](#)

Counting Sort (when range of values is small)

What it is: Instead of comparing elements to each other, just count how many times each value appears (works only when values are small integers in a known range), then rebuild the array from those counts.

Recognize it by: values are small non-negative integers within a known, limited range (e.g. ages, marks out of 100).

Problem: Sort an array of non-negative integers whose maximum value is small and known in advance.

Example: `a = [4,2,2,8,3,3,1]`, `maxVal = 8` → Output: `[1,2,2,3,3,4,8]`

```
void countingSort(vector<int>& a, int maxVal) {
    vector<int> count(maxVal + 1, 0);
    for (int x : a) count[x]++;
    int idx = 0;
    for (int v = 0; v <= maxVal; v++)
        while (count[v]-- > 0) a[idx++] = v;
}
```

Time: $O(n + k)$. **Space:** $O(k)$.

Practice: [Sort an Array](#) · [Sort Colors](#)

In interviews, `sort(a.begin(), a.end())` (introsort, $O(n \log n)$) is usually fine — know the theory, use STL in practice.

6. Binary Search Patterns

Intuition: Works whenever the search space is *monotonic* — i.e., there's a point where a condition flips from false→true (or true→false). Not just for sorted arrays; also for "search on the answer".

6a. Standard binary search

What it is: Repeatedly look at the middle element of a sorted array and eliminate the half that can't contain the target — cuts the search space in half every step instead of scanning one by one.

Recognize it by: the array (or search space) is already sorted and you need to find one exact value quickly.

Problem: Given a sorted array and a target value, return the index of the target, or -1 if it isn't present.

Example: `a = [-1,0,3,5,9,12]`, `target = 9` → Output: 4

```
int binarySearch(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Time: $O(\log n)$. **Space:** $O(1)$.

Practice: [Binary Search](#)

6b. First / Last occurrence (lower_bound / upper_bound style)

What it is: A tweak on standard binary search: even after finding a match, keep searching further left (or right) to find the very first (or last) position where the target occurs, since duplicates may exist.

Recognize it by: sorted array WITH duplicates, and you need the first or last position of a value, or a count of occurrences.

Problem: Given a sorted array that may contain duplicates, find the first (or last) index at which a target value occurs.

Example: $a = [5, 7, 7, 8, 8, 10]$, $\text{target} = 8 \rightarrow \text{Output: first index} = 3$

```
int firstOccurrence(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { ans = mid; hi = mid - 1; } // keep searching left
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return ans;
}
```

Time: $O(\log n)$. **Space:** $O(1)$.

Practice: [Find First and Last Position of Element in Sorted Array](#)

6c. Search in rotated sorted array

What it is: The array was sorted but then "rotated" (its front chunk moved to the back). At every step, one half of the current range is still normally sorted — figure out which half that is, then decide which side to continue searching in.

Recognize it by: the phrase "rotated sorted array" or "sorted array that has been shifted", with no duplicates mentioned.

Intuition: One half of the array (around mid) is always properly sorted — check which half, then decide which side to recurse into.

Problem: A sorted array has been rotated at some unknown pivot (e.g. $[0, 1, 2, 4, 5, 6, 7] \rightarrow [4, 5, 6, 7, 0, 1, 2]$). Given the rotated array and a target, find its index.

Example: $a = [4, 5, 6, 7, 0, 1, 2]$, $\text{target} = 0 \rightarrow \text{Output: } 4$

Dry Run:

```
lo=0 hi=6 mid=3 a[mid]=7 | left half [0..3] is sorted & target NOT in it -> lo=4
lo=4 hi=6 mid=5 a[mid]=1 | left half [4..5] is sorted & target in it -> hi=4
lo=4 hi=4 mid=4 a[mid]=0 -> found at 4
```

```
int searchRotated(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) { // left half sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else { // right half sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Time: $O(\log n)$. **Space:** $O(1)$.

Practice: [Search in Rotated Sorted Array](#) · [Search in Rotated Sorted Array II](#) · [Find Minimum in Rotated Sorted Array](#)

6d. Binary Search on the Answer

What it is: Instead of searching through the array, binary search over the space of POSSIBLE ANSWERS (e.g. "could the answer be 5? could it be 10?") whenever "is this answer good enough?" gets easier to check as the guessed value increases.

Intuition: When the answer itself is a number in a range, and "can we achieve X?" is monotonic (feasible answers form a prefix/suffix), binary search over the answer value.

Problem: You're given `bloomDay[i]` (the day flower *i* blooms), and need *m* bouquets, each requiring *k adjacent* already-bloomed flowers. Find the minimum day on which you can make all *m* bouquets (or -1 if impossible).

Example: `bloomDay = [1,10,3,10,2]`, *m* = 3, *k* = 1 → Output: 3

Dry Run:

```
lo=1 hi=10 try day=5 -> canMake=True -> shrink hi=5
lo=1 hi=5 try day=3 -> canMake=True -> shrink hi=3
lo=1 hi=3 try day=2 -> canMake=False -> raise lo=3
Answer: 3
```

```
// Example: minimum days to make m bouquets of k adjacent flowers
bool canMake(vector<int>& bloom, int day, int m, int k) {
    int count = 0, bouquets = 0;
    for (int b : bloom) {
        if (b <= day) count++;
        else count = 0;
        if (count == k) { bouquets++; count = 0; }
    }
    return bouquets >= m;
}

int minDays(vector<int>& bloom, int m, int k) {
    if ((long long)m * k > bloom.size()) return -1;
    int lo = *min_element(bloom.begin(), bloom.end());
    int hi = *max_element(bloom.begin(), bloom.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canMake(bloom, mid, m, k)) hi = mid; // try smaller
        else lo = mid + 1;
    }
    return lo;
}
```

Time: $O(n \log(\max - \min))$. **Space:** $O(1)$.

Practice: [Koko Eating Bananas](#) · [Capacity To Ship Packages Within D Days](#) · [Split Array Largest Sum](#) **Recognize this pattern from:** "minimize the maximum", "maximize the minimum", "Koko eating bananas", "ship packages within D days", "split array largest sum".

7. Recursion & Backtracking

Intuition: Backtracking = try a choice → recurse → undo the choice (backtrack) if it doesn't lead to a solution. Think of it as DFS over a decision tree, pruning invalid branches early.

Template:

```
void backtrack(/* state */) {
    if (/* base case: found a solution */) {
        // record it
        return;
    }
    for (/* each choice */) {
```

```

        if (/* choice invalid */) continue;    // pruning
        // make choice
        backtrack(/* updated state */);
        // undo choice
    }
}

```

Subsets (power set)

What it is: For every element, branch into two choices — include it, or don't — building every possible combination. Backtracking explores one full choice-path at a time, then undoes the last choice to try the next option.

Recognize it by: "return all subsets", "all possible combinations", "power set" — order doesn't matter, size varies.

Problem: Given an array of distinct integers, return every possible subset (the power set).

Example: nums = [1,2,3] → Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Dry Run:

```

record subset []
choose 1 -> cur=[1]
    record subset [1]
    choose 2 -> cur=[1, 2]
        record subset [1, 2]
        choose 3 -> cur=[1, 2, 3]
            record subset [1, 2, 3]
            backtrack: remove 3 -> cur=[1, 2]
        backtrack: remove 2 -> cur=[1]
    choose 3 -> cur=[1, 3]
        record subset [1, 3]
    backtrack: remove 3 -> cur=[1]
backtrack: remove 1 -> cur=[]
choose 2 -> cur=[2]
    record subset [2]
    choose 3 -> cur=[2, 3]
        record subset [2, 3]
    backtrack: remove 3 -> cur=[2]
backtrack: remove 2 -> cur=[]
choose 3 -> cur=[3]
    record subset [3]
backtrack: remove 3 -> cur=[]
Total subsets found: 8

```

```

void subsetsHelper(vector<int>& nums, int idx, vector<int>& cur, vector<vector<int>>& res) {
    res.push_back(cur);
    for (int i = idx; i < nums.size(); i++) {
        cur.push_back(nums[i]);
        subsetsHelper(nums, i + 1, cur, res);
        cur.pop_back();          // backtrack
    }
}

```

Time: $O(2^n \cdot n)$ (2^n subsets, $O(n)$ to copy each). **Space:** $O(n)$ recursion depth (+ $O(2^n \cdot n)$ output).

Practice: [Subsets](#) · [Subsets II](#)

Permutations

What it is: Try placing each unused element next in the sequence, recurse to fill the remaining positions, then undo ("backtrack") and try a different element in that position — this generates every possible ordering.

Recognize it by: "return all orderings/arrangements", every element must be used exactly once and order matters.

Problem: Given an array of distinct integers, return every possible ordering (permutation) of them.

Example: nums = [1,2,3] → Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Dry Run:

```
choose 1 -> cur=[1]
  choose 2 -> cur=[1, 2]
    choose 3 -> cur=[1, 2, 3]
      complete permutation: [1, 2, 3]
  choose 3 -> cur=[1, 3]
    choose 2 -> cur=[1, 3, 2]
      complete permutation: [1, 3, 2]
choose 2 -> cur=[2]
  choose 1 -> cur=[2, 1]
    choose 3 -> cur=[2, 1, 3]
      complete permutation: [2, 1, 3]
  choose 3 -> cur=[2, 3]
    choose 1 -> cur=[2, 3, 1]
      complete permutation: [2, 3, 1]
choose 3 -> cur=[3]
  choose 1 -> cur=[3, 1]
    choose 2 -> cur=[3, 1, 2]
      complete permutation: [3, 1, 2]
  choose 2 -> cur=[3, 2]
    choose 1 -> cur=[3, 2, 1]
      complete permutation: [3, 2, 1]
Total permutations found: 6
```

```
void permuteHelper(vector<int>& nums, vector<bool>& used, vector<int>& cur, vector<vector<int>>& res) {
    if (cur.size() == nums.size()) { res.push_back(cur); return; }
    for (int i = 0; i < nums.size(); i++) {
        if (used[i]) continue;
        used[i] = true;
        cur.push_back(nums[i]);
        permuteHelper(nums, used, cur, res);
        cur.pop_back();
        used[i] = false;           // backtrack
    }
}
```

Time: $O(n! \cdot n)$. **Space:** $O(n)$.

Practice: [Permutations](#) · [Permutations II](#)

N-Queens

What it is: Place one queen per row; before placing, check if it's under attack from queens already placed. If a placement leads to a dead end later, backtrack and try the next column in that row.

Recognize it by: placing items on a grid under "no two can conflict" constraints (rows/columns/diagonals), needing every valid full arrangement.

Problem: Place n queens on an $n \times n$ chessboard so that no two queens attack each other (same row, column, or diagonal); return all distinct arrangements.

Example: $n = 4$ → Output: 2 valid board arrangements

Dry Run:

```

row 0: try col 0 (not attacked) -> place, board=[0, -1, -1, -1]
row 1: col 0 is attacked by an earlier queen -> skip
row 1: col 1 is attacked by an earlier queen -> skip
row 1: try col 2 (not attacked) -> place, board=[0, 2, -1, -1]
row 1: try col 3 (not attacked) -> place, board=[0, 3, -1, -1]
row 2: try col 1 (not attacked) -> place, board=[0, 3, 1, -1]
row 0: try col 1 (not attacked) -> place, board=[1, -1, -1, -1]
row 1: col 0 is attacked by an earlier queen -> skip
row 1: col 1 is attacked by an earlier queen -> skip
row 1: col 2 is attacked by an earlier queen -> skip
row 1: try col 3 (not attacked) -> place, board=[1, 3, -1, -1]
row 2: try col 0 (not attacked) -> place, board=[1, 3, 0, -1]
row 3: try col 2 (not attacked) -> place, board=[1, 3, 0, 2]
row 4: all queens placed -> first solution = [1, 3, 0, 2]
... continues exploring remaining branches ...
Answer: 2 total solutions for n=4

```

```

bool isSafe(vector<string>& board, int row, int col, int n) {
    for (int i = 0; i < row; i++) if (board[i][col] == 'Q') return false;
    for (int i = row-1, j = col-1; i >= 0 && j >= 0; i--, j--) if (board[i][j]=='Q') return false;
    for (int i = row-1, j = col+1; i >= 0 && j < n; i--, j++) if (board[i][j]=='Q') return false;
    return true;
}

void solveNQueens(vector<string>& board, int row, int n, vector<vector<string>>& res) {
    if (row == n) { res.push_back(board); return; }
    for (int col = 0; col < n; col++) {
        if (!isSafe(board, row, col, n)) continue;
        board[row][col] = 'Q';
        solveNQueens(board, row + 1, n, res);
        board[row][col] = '.'; // backtrack
    }
}

```

Time: $O(n!)$ roughly (with pruning much faster in practice). **Space:** $O(n^2)$ for board + $O(n)$ recursion.

Practice: [N-Queens](#) · [Sudoku Solver](#) · [Combination Sum](#) · [Word Search](#) · [Palindrome Partitioning](#)

Backtracking recognizes: subsets, combinations, permutations, N-Queens, Sudoku solver, word search on grid, palindrome partitioning, combination sum.

8. Linked List Patterns

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

```

Reverse a linked list (iterative)

What it is: Walk through the list one node at a time, and instead of pointing each node to the NEXT node, make it point to the PREVIOUS one — by the end, the whole list points backward.

Recognize it by: "reverse a linked list" (whole list or a sub-range of it) — a classic pointer-rewiring task.

Intuition: Walk the list, re-point each **next** backward, carry three pointers: prev, curr, next.

Problem: Given the head of a singly linked list, reverse it and return the new head.

Example: 1→2→3→4→5 → Output: 5→4→3→2→1

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev; // new head
}
```

Time: O(n). **Space:** O(1).

Practice: [Reverse Linked List](#) · [Reverse Linked List II](#)

Floyd's Cycle Detection (slow/fast pointers)

What it is: Send two pointers down the list at different speeds (one step vs two steps at a time). If there's a loop, the faster one will eventually "lap" the slower one and they'll meet — like two runners on a circular track.

Recognize it by: "does this linked list have a cycle", "find the start of the cycle", or any problem framed as "detect a loop".

Intuition: Fast pointer moves 2 steps, slow moves 1. If there's a cycle, they must eventually meet (fast "laps" slow).

Problem: Given the head of a linked list, determine whether it contains a cycle (some node's **next** eventually points back to an earlier node).

Example: 3→2→0→4→(points back to the node valued 2) → Output: true

```
bool hasCycle(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Finding cycle start: after slow==fast meet, reset one pointer to head; move both one step at a time — they meet at cycle start.

```
ListNode* detectCycleStart(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            ListNode* ptr = head;
            while (ptr != slow) { ptr = ptr->next; slow = slow->next; }
            return ptr;
        }
    }
    return nullptr;
}
```

Time: O(n). **Space:** O(1).

Practice: [Linked List Cycle](#) · [Linked List Cycle II](#) · [Happy Number](#)

Merge two sorted lists

What it is: Walk both lists side by side with two pointers, always picking whichever current node is smaller to attach next to the result — since both inputs are already sorted, the merged result comes out sorted too.

Recognize it by: combining two (or more) already-sorted linked lists or arrays into one sorted result.

Problem: Given the heads of two sorted linked lists, merge them into one sorted linked list.

Example: list1 = 1→2→4, list2 = 1→3→4 → Output: 1→1→2→3→4→4

```
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}
```

Time: $O(n+m)$. **Space:** $O(1)$ (excluding output).

Practice: [Merge Two Sorted Lists](#)

Find middle of linked list (slow/fast)

What it is: Move one pointer one step at a time and another pointer two steps at a time; when the fast one reaches the end, the slow one is sitting exactly at the middle.

Recognize it by: you need the middle node of a linked list without knowing its length in advance (common inside merge-sort-on-list or palindrome-list problems).

Problem: Given the head of a linked list, return its middle node (if there are two middle nodes, return the second one).

Example: 1→2→3→4→5 → Output: node with value 3

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
```

Time: $O(n)$. **Space:** $O(1)$.

Practice: [Middle of the Linked List](#) · [Palindrome Linked List](#) Used inside merge sort on linked lists, and palindrome-linked-list checks.

LRU Cache (linked list + hashmap)

What it is: Combine a doubly linked list (which remembers the ORDER items were used, most-recent at the front) with a hashmap (which gives instant lookup by key) to build a cache that can evict the least-recently-used item in $O(1)$ time.

Recognize it by: "design a cache", "evict least recently used", any problem asking for $O(1)$ get/put with an eviction policy.

Intuition: Doubly linked list keeps usage order (most-recent at head); hashmap gives $O(1)$ node lookup by key.

Problem: Design a fixed-capacity cache supporting `get(key)` and `put(key, value)` in $O(1)$ time each, that evicts the Least Recently Used item whenever it's full.

Example: capacity=2; `put(1,1)`; `put(2,2)`; `get(1)`→1; `put(3,3)` evicts key 2; `get(2)`→-1

```
class LRUCache {
    int cap;
    list<pair<int,int>> dll; // {key, value}, front = most recently used
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        if (!mp.count(key)) return -1;
        auto it = mp[key];
        dll.splice(dll.begin(), dll, it); // move to front
        return it->second;
    }
    void put(int key, int value) {
        if (mp.count(key)) {
            dll.erase(mp[key]);
        } else if (dll.size() == cap) {
            mp.erase(dll.back().first);
            dll.pop_back();
        }
        dll.push_front({key, value});
        mp[key] = dll.begin();
    }
};
```

Time: $O(1)$ get/put. **Space:** $O(\text{capacity})$.

Practice: [LRU Cache](#) · [LFU Cache](#)

9. Stack Patterns

Valid Parentheses

What it is: Use a stack as a "pile" of open brackets waiting to be closed: every opening bracket gets pushed on, and every closing bracket must match (and pop) whatever is currently on top — otherwise the brackets don't line up correctly.

Recognize it by: matching/nesting of paired symbols — brackets, tags, or any "open X must be closed by a matching Y" rule.

Problem: Given a string containing just the characters '(', ')', '{', '}', '[', ']', determine whether every bracket is properly closed and nested.

Example: `"()[]{}"` → Output: true. `"[()]"` → Output: false

```
bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> match = {{ '}', '{' }, { ')', '(' }, { ']', '[' }, { '}', '{' }, { '}', '{' } };
    for (char c : s) {
        if (c=='(' || c=='[' || c=='{') st.push(c);
        else {
            if (st.empty() || st.top() != match[c]) return false;
            st.pop();
        }
    }
}
```

```

    }
    return st.empty();
}

```

Time: $O(n)$. **Space:** $O(n)$.

Practice: [Valid Parentheses](#) · [Min Stack](#)

Monotonic Stack — Next Greater Element

What it is: Keep a stack of elements that are waiting to find something bigger than them. As soon as a bigger element shows up, it "resolves" every smaller element still sitting on the stack, all in one linear pass.

Recognize it by: "next greater/smaller element", "daily temperatures", "stock span" — anything needing, for each element, the nearest bigger/smaller one on one side.

Intuition: Keep a stack of indices whose "next greater" hasn't been found yet. When a bigger element comes, it's the answer for everything smaller left on the stack.

Problem: For every element in an array, find the first element to its right that is strictly greater; -1 if none exists.

Example: $a = [2, 1, 2, 4, 3] \rightarrow \text{Output: } [4, 2, 4, -1, -1]$

Dry Run:

```

i=0 a[i]=2  stack(idx)=[]
    push 0 -> stack=[0]
i=1 a[i]=1  stack(idx)=[0]
    push 1 -> stack=[0, 1]
i=2 a[i]=2  stack(idx)=[0, 1]
    a[1]=1 < 2 -> res[1]=2, pop 1
    push 2 -> stack=[0, 2]
i=3 a[i]=4  stack(idx)=[0, 2]
    a[2]=2 < 4 -> res[2]=4, pop 2
    a[0]=2 < 4 -> res[0]=4, pop 0
    push 3 -> stack=[3]
i=4 a[i]=3  stack(idx)=[3]
    push 4 -> stack=[3, 4]
Answer: [4, 2, 4, -1, -1]

```

```

vector<int> nextGreaterElement(vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st; // stores indices, values decreasing bottom-top
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            res[st.top()] = a[i];
            st.pop();
        }
        st.push(i);
    }
    return res;
}

```

Time: $O(n)$ — each index pushed & popped at most once. **Space:** $O(n)$.

Practice: [Next Greater Element I](#) · [Daily Temperatures](#) · [Online Stock Span](#)

Same pattern powers: largest rectangle in histogram, daily temperatures, stock span, trapping rain water (stack variant).

Largest Rectangle in Histogram (monotonic stack)

What it is: Keep a stack of bar heights in increasing order. When a shorter bar appears, it means the taller bars behind it can't extend any further right, so pop them off and calculate the biggest rectangle they could have formed.

Recognize it by: "largest rectangle", "maximal rectangle", or trapping-water style problems built on top of a next-greater-element idea.

Problem: Given the heights of histogram bars (each of width 1), find the area of the largest rectangle that can be formed within the histogram.

Example: heights = [2,1,5,6,2,3] → Output: 10

Dry Run:

```
i=0 cur=2  stack(idx)=[ ]
i=1 cur=1  stack(idx)=[0]
  pop height=2 width=1 area=2 -> best=2
i=2 cur=5  stack(idx)=[1]
i=3 cur=6  stack(idx)=[1, 2]
i=4 cur=2  stack(idx)=[1, 2, 3]
  pop height=6 width=1 area=6 -> best=6
  pop height=5 width=2 area=10 -> best=10
i=5 cur=3  stack(idx)=[1, 4]
i=6 cur=0  stack(idx)=[1, 4, 5]
  pop height=3 width=1 area=3 -> best=10
  pop height=2 width=4 area=8 -> best=10
  pop height=1 width=6 area=6 -> best=10
Answer: 10
```

```
int largestRectangleArea(vector<int>& h) {
    stack<int> st; // increasing heights
    int best = 0, n = h.size();
    for (int i = 0; i <= n; i++) {
        int cur = (i == n) ? 0 : h[i];
        while (!st.empty() && h[st.top()] >= cur) {
            int height = h[st.top()]; st.pop();
            int width = st.empty() ? i : i - st.top() - 1;
            best = max(best, height * width);
        }
        st.push(i);
    }
    return best;
}
```

Time: $O(n)$. **Space:** $O(n)$.

Practice: [Largest Rectangle in Histogram](#) · [Trapping Rain Water](#) · [Maximal Rectangle](#)

10. Queue & Deque Patterns

Sliding Window Maximum (monotonic deque)

What it is: Keep a double-ended queue (deque) of candidates for "current window's maximum", always removing smaller elements from the back before adding a new one — so the front of the deque is always the answer for the current window.

Recognize it by: "maximum/minimum in every window of size k" — a sliding window where you need the extreme value, not a sum.

Intuition: Maintain a deque of indices with strictly decreasing values (front = current window max). Pop smaller values from the back before pushing; pop from front if out of window.

Problem: Given an array and a window size k , return the maximum value in every window of size k as it slides from left to right.

Example: $a = [1, 3, -1, -3, 5, 3, 6, 7]$, $k = 3 \rightarrow$ Output: $[3, 3, 5, 5, 6, 7]$

Dry Run:

```
i=0: deque(idx)=[0] -> window not full yet
i=1: a[0]=1 < 3 -> pop_back
i=1: deque(idx)=[1] -> window not full yet
i=2: deque(idx)=[1, 2] -> window max = 3
i=3: deque(idx)=[1, 2, 3] -> window max = 3
i=4: front idx 1 out of window -> pop_front
i=4: a[3]=-3 < 5 -> pop_back
i=4: a[2]=-1 < 5 -> pop_back
i=4: deque(idx)=[4] -> window max = 5
i=5: deque(idx)=[4, 5] -> window max = 5
i=6: a[5]=3 < 6 -> pop_back
i=6: a[4]=5 < 6 -> pop_back
i=6: deque(idx)=[6] -> window max = 6
i=7: a[6]=6 < 7 -> pop_back
i=7: deque(idx)=[7] -> window max = 7
Answer: [3, 3, 5, 5, 6, 7]
```

```
vector<int> maxSlidingWindow(vector<int>& a, int k) {
    deque<int> dq; // stores indices
    vector<int> res;
    for (int i = 0; i < a.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // out of window
        while (!dq.empty() && a[dq.back()] < a[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

Time: $O(n)$. **Space:** $O(k)$.

Practice: [Sliding Window Maximum](#)

BFS with a queue (generic template)

What it is: Explore a graph or grid "layer by layer": visit all neighbours of the start node first, then all of THEIR neighbours, and so on — using a queue guarantees you finish one full layer before starting the next, which is what gives BFS its shortest-path guarantee.

Recognize it by: "shortest path", "minimum number of steps", "level by level" in an unweighted graph or grid.

Problem: Given a 2D grid of '1' (land) and '0' (water), count the number of islands (groups of connected land cells).

Example: $grid = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \rightarrow$ Output: 2

```
void bfs(int start, vector<vector<int>>& adj, vector<bool>& visited) {
    queue<int> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        // process node
    }
}
```

```

        for (int nxt : adj[node]) {
            if (!visited[nxt]) {
                visited[nxt] = true;
                q.push(nxt);
            }
        }
    }
}

```

Time: $O(V+E)$. **Space:** $O(V)$.

Practice: [Number of Islands](#) · [Rotting Oranges](#)

11. Trees

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

```

Traversals

What it is: The different orders in which you can visit every node of a tree: top-first (preorder), sorted order for a BST (inorder), children-before-parent (postorder), or level-by-level using a queue (level order / BFS).

Recognize it by: any question that starts with "given a binary tree, return/print/visit its nodes in ____ order".

Problem: Given the root of a binary tree, return its node values visited in a given order (preorder, inorder, postorder, or level order).

Example: tree = [1,null,2,3] → inorder Output: [1,3,2]

```

void preorder(TreeNode* root, vector<int>& out) { // Root-Left-Right
    if (!root) return;
    out.push_back(root->val);
    preorder(root->left, out);
    preorder(root->right, out);
}

void inorder(TreeNode* root, vector<int>& out) { // Left-Root-Right (sorted for BST)
    if (!root) return;
    inorder(root->left, out);
    out.push_back(root->val);
    inorder(root->right, out);
}

void postorder(TreeNode* root, vector<int>& out) { // Left-Right-Root
    if (!root) return;
    postorder(root->left, out);
    postorder(root->right, out);
    out.push_back(root->val);
}

vector<vector<int>>> levelOrder(TreeNode* root) { // BFS level by level
    vector<vector<int>>> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
        }
    }
}

```

```

        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
    res.push_back(level);
}
return res;
}

```

Time: $O(n)$ all traversals. **Space:** $O(h)$ for DFS recursion (h = height), $O(n)$ worst case for BFS queue.

Practice: [Binary Tree Inorder Traversal](#) · [Binary Tree Level Order Traversal](#) · [Binary Tree Zigzag Level Order Traversal](#)

Height / Diameter of a tree

What it is: Compute a value "bottom-up": each node asks its children for their answers first (via recursion), combines them, and passes its own answer up to its parent — the classic recursive tree pattern.

Recognize it by: "height of a tree", "diameter" (longest path between two nodes), "is this tree balanced" — needs a value computed bottom-up from children.

Intuition: Diameter (longest path between any two nodes) at any node = left height + right height. Compute bottom-up.

Problem: Given the root of a binary tree, find the length (in edges) of the longest path between any two nodes.

Example: tree = [1,2,3,4,5] → Output: diameter = 3

```

int diameter = 0;
int height(TreeNode* root) {
    if (!root) return 0;
    int lh = height(root->left);
    int rh = height(root->right);
    diameter = max(diameter, lh + rh); // update global answer
    return 1 + max(lh, rh);
}

```

Time: $O(n)$. **Space:** $O(h)$.

Practice: [Maximum Depth of Binary Tree](#) · [Diameter of Binary Tree](#) · [Balanced Binary Tree](#)

Lowest Common Ancestor (general binary tree)

What it is: Search both subtrees for the two target nodes. If they turn up in DIFFERENT subtrees of some node, that node is where their paths first meet — the lowest common ancestor.

Recognize it by: "find the lowest/nearest common ancestor of two nodes" in a tree.

Intuition: Recurse into both subtrees; if a node finds both p and q in different subtrees, it IS the LCA.

Problem: Given the root of a binary tree and two of its nodes p and q , find their lowest (deepest) common ancestor.

Example: tree with $p = 5$, $q = 1$ → Output: their LCA is the node valued 3

```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if (!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if (left && right) return root; // p and q found in different subtrees
    return left ? left : right;
}

```

Time: $O(n)$. **Space:** $O(h)$.

Practice: [Lowest Common Ancestor of a Binary Tree](#)

Serialize / Deserialize

What it is: "Serialize" turns a tree into a flat string (e.g. using preorder traversal, marking empty spots with a placeholder like '#') so it can be saved or sent somewhere; "deserialize" reads that string back and rebuilds the exact same tree structure.

Recognize it by: "design an algorithm to serialize/deserialize" a tree — converting a structure to a string and back losslessly.

Problem: Design an algorithm to convert a binary tree into a string (serialize), and reconstruct the exact same tree from that string (deserialize).

Example: tree = [1,2,3,null,null,4,5] → serialized: "1 2 # # 3 4 # # 5 # #"

```
void serialize(TreeNode* root, string& out) {
    if (!root) { out += "# "; return; }
    out += to_string(root->val) + " ";
    serialize(root->left, out);
    serialize(root->right, out);
}

TreeNode* deserializeHelper(istringstream& in) {
    string val; in >> val;
    if (val == "#") return nullptr;
    TreeNode* root = new TreeNode(stoi(val));
    root->left = deserializeHelper(in);
    root->right = deserializeHelper(in);
    return root;
}
```

Time: $O(n)$. **Space:** $O(n)$.

Practice: [Serialize and Deserialize Binary Tree](#)

12. Binary Search Trees

BST Insert / Search

What it is: Because a Binary Search Tree keeps smaller values to the left and bigger values to the right of every node, you can decide which direction to go at each step just by comparing — no need to check every node.

Recognize it by: operations on a Binary Search Tree where you must decide left/right using value comparisons instead of checking every node.

Problem: Given the root of a Binary Search Tree, insert a new value into it (or search for whether a value exists), keeping the BST property intact.

Example: insert 5 into BST rooted at 4 → 5 becomes the right child of 4

```
TreeNode* insertBST(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val) root->left = insertBST(root->left, val);
    else root->right = insertBST(root->right, val);
    return root;
}
```

```
TreeNode* searchBST(TreeNode* root, int val) {
    if (!root || root->val == val) return root;
    return val < root->val ? searchBST(root->left, val) : searchBST(root->right, val);
}
```

Time: $O(h)$ — $O(\log n)$ balanced, $O(n)$ skewed worst case. **Space:** $O(h)$.

Practice: [Insert into a Binary Search Tree](#) · [Search in a Binary Search Tree](#)

BST Delete

What it is: Removing a node is easy if it has 0 or 1 children (just reconnect around it), but if it has 2 children, you swap its value with the next-bigger value in the tree (its "inorder successor") and delete that instead.

Recognize it by: "delete a node from a BST" while keeping the BST property valid.

Intuition: 3 cases — leaf (just remove), one child (replace with child), two children (replace with inorder successor, i.e., min of right subtree, then delete that).

Problem: Given the root of a BST and a key, delete the node with that key from the tree while keeping it a valid BST.

Example: delete 3 from [5,3,6,2,4,null,7] → Output: [5,4,6,2,null,null,7]

```
TreeNode* deleteNode(TreeNode* root, int key) {
    if (!root) return nullptr;
    if (key < root->val) root->left = deleteNode(root->left, key);
    else if (key > root->val) root->right = deleteNode(root->right, key);
    else {
        if (!root->left) return root->right;
        if (!root->right) return root->left;
        TreeNode* succ = root->right;
        while (succ->left) succ = succ->left; // inorder successor
        root->val = succ->val;
        root->right = deleteNode(root->right, succ->val);
    }
    return root;
}
```

Time: $O(h)$. **Space:** $O(h)$.

Practice: [Delete Node in a BST](#)

Validate BST

What it is: Check that every node's value actually falls within the valid (min, max) range allowed by its ancestors — just checking a node against its immediate parent isn't enough, since a value could break the rule set by a grandparent further up.

Recognize it by: "determine if a binary tree is a valid BST" — check ordering constraints across the WHOLE ancestor chain, not just parent-child.

Problem: Given the root of a binary tree, determine whether it is a valid Binary Search Tree.

Example: [5,1,4,null,null,3,6] → Output: false (4's left child 3 is less than 5, breaking the BST rule two levels up)

```
bool validate(TreeNode* root, long long lo, long long hi) {
    if (!root) return true;
    if (root->val <= lo || root->val >= hi) return false;
    return validate(root->left, lo, root->val) && validate(root->right, root->val, hi);
}

bool isValidBST(TreeNode* root) {
    return validate(root, LLONG_MIN, LLONG_MAX);
}
```

}

Time: $O(n)$. **Space:** $O(h)$.**Practice:** [Validate Binary Search Tree](#)

Kth Smallest in BST (inorder gives sorted order)

What it is: An inorder traversal of a BST always visits nodes in sorted order — so simply count nodes as you traverse inorder, and stop as soon as you reach the kth one.

Recognize it by: "kth smallest/largest element in a BST" — inorder traversal gives you sorted order for free.

Problem: Given the root of a BST and an integer k, return the k-th smallest value stored in it.

Example: [3,1,4,null,2], k = 1 → Output: 1

```
int kthSmallest(TreeNode* root, int k) {
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (true) {
        while (cur) { st.push(cur); cur = cur->left; }
        cur = st.top(); st.pop();
        if (--k == 0) return cur->val;
        cur = cur->right;
    }
}
```

Time: $O(h + k)$. **Space:** $O(h)$.**Practice:** [Kth Smallest Element in a BST](#)

13. Heaps / Priority Queue

Intuition: A heap gives $O(\log n)$ insert and $O(1)$ access to min/max. Use whenever you repeatedly need "the current smallest/largest" — top-K problems, merging sorted lists, scheduling.

```
priority_queue<int> maxHeap; // default: max at top
priority_queue<int, vector<int>, greater<int>> minHeap; // min at top
```

Kth Largest Element

What it is: Keep a min-heap (a structure that gives instant access to the smallest element) of size k as you scan the array. Whenever the heap grows past k, kick out its smallest — whatever survives at the top is the kth largest.

Recognize it by: "kth largest/smallest", "top k" elements from an unsorted collection.

Intuition: Keep a min-heap of size k. If heap grows beyond k, pop smallest — heap top ends up as the kth largest.

Problem: Given an unsorted array and an integer k, find the k-th largest element in the array.

Example: nums = [3,2,1,5,6,4], k = 2 → Output: 5

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : nums) {
        minHeap.push(x);
        if (minHeap.size() > k) minHeap.pop();
    }
}
```

```
    return minHeap.top();
}
```

Time: $O(n \log k)$. **Space:** $O(k)$.

Practice: [Kth Largest Element in an Array](#) · [Find Median from Data Stream](#)

Merge K Sorted Lists

What it is: Put the front node of every list into a min-heap. Repeatedly pull out the smallest, attach it to your result, and push in whatever node came right after it from the same list — the heap always knows the next-smallest across ALL lists.

Recognize it by: merging MORE THAN TWO sorted lists/arrays at once — a heap generalizes the two-pointer merge to k inputs.

Problem: Given an array of k sorted linked lists, merge them all into one sorted linked list.

Example: lists = [[1,4,5],[1,3,4],[2,6]] → Output: [1,1,2,3,4,4,5,6]

```
struct Compare {
    bool operator()(ListNode* a, ListNode* b) { return a->val > b->val; }
};
ListNode* mergeKLists(vector<ListNode*>& lists) {
    priority_queue<ListNode*, vector<ListNode*>, Compare> pq;
    for (auto l : lists) if (l) pq.push(l);
    ListNode dummy(0), *tail = &dummy;
    while (!pq.empty()) {
        ListNode* node = pq.top(); pq.pop();
        tail->next = node; tail = node;
        if (node->next) pq.push(node->next);
    }
    return dummy.next;
}
```

Time: $O(n \log k)$ where n = total nodes, k = number of lists. **Space:** $O(k)$.

Practice: [Merge k Sorted Lists](#)

Top-K Frequent Elements (heap + hashmap)

What it is: First count how often each element appears using a hashmap, then use a small heap of size k to keep track of only the k most frequent ones seen so far.

Recognize it by: "k most/least frequent elements", "k closest points" — top-k by some computed score.

Problem: Given an integer array and an integer k, return the k most frequently occurring elements.

Example: nums = [1,1,1,2,2,3], k = 2 → Output: [1,2]

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    for (int x : nums) freq[x]++;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> minHeap; // {freq, val}
    for (auto& [val, f] : freq) {
        minHeap.push({f, val});
        if (minHeap.size() > k) minHeap.pop();
    }
    vector<int> res;
    while (!minHeap.empty()) { res.push_back(minHeap.top().second); minHeap.pop(); }
    return res;
}
```

Time: $O(n \log k)$. **Space:** $O(n)$.

Practice: [Top K Frequent Elements](#) · [K Closest Points to Origin](#)

14. Tries

Intuition: A tree where each edge is a character; words sharing prefixes share paths. Enables $O(\text{word length})$ prefix/search operations instead of scanning all words.

```
struct TrieNode {
    TrieNode* children[26] = {};
    bool isEnd = false;
};

class Trie {
    TrieNode* root;
public:
    Trie() { root = new TrieNode(); }

    void insert(string word) {
        TrieNode* node = root;
        for (char c : word) {
            int idx = c - 'a';
            if (!node->children[idx]) node->children[idx] = new TrieNode();
            node = node->children[idx];
        }
        node->isEnd = true;
    }

    bool search(string word) {
        TrieNode* node = find(word);
        return node && node->isEnd;
    }

    bool startsWith(string prefix) {
        return find(prefix) != nullptr;
    }

private:
    TrieNode* find(string s) {
        TrieNode* node = root;
        for (char c : s) {
            int idx = c - 'a';
            if (!node->children[idx]) return nullptr;
            node = node->children[idx];
        }
        return node;
    }
};
```

Time: $O(L)$ per insert/search where L = word length. **Space:** $O(\text{total characters inserted} \times 26)$ worst case (or use `unordered_map<char, TrieNode*>` for sparser storage).

Used for: autocomplete, word search II (backtracking + trie), longest common prefix, IP routing.

15. Graphs

Representation:

```
vector<vector<int>>> adj(n); // unweighted
adj[u].push_back(v); adj[v].push_back(u); // undirected

vector<vector<pair<int,int>>> wadj(n); // weighted: {neighbor, weight}
wadj[u].push_back({v, w});
```

DFS (recursive)

What it is: Go as deep as possible down one path before backtracking — dive into a neighbour, then that neighbour's neighbour, and so on, only coming back up once you hit a dead end.

Recognize it by: "count connected components / islands", "explore as far as possible", or any grid/graph problem naturally expressed with recursion.

Problem: Given a 2D grid of '1' (land) and '0' (water), count the number of islands using depth-first search.

Example: grid = `[["1","1","0"],["0","1","0"],["0","0","1"]]` → Output: 2

```
void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    // process node
    for (int nxt : adj[node])
        if (!visited[nxt]) dfs(nxt, adj, visited);
}
```

Time: $O(V+E)$. **Space:** $O(V)$ recursion + visited array.

Practice: [Number of Islands](#) · [Clone Graph](#)

BFS — shortest path in unweighted graph

What it is: Explore the graph in expanding "rings" outward from the start node using a queue. Because you finish visiting everything at distance 1 before moving to distance 2, the first time you reach any node is via the shortest path.

Recognize it by: "fewest steps/moves", "shortest transformation sequence", "minimum number of operations" in an unweighted graph.

Problem: Given an unweighted graph and a source node, find the shortest distance (in number of edges) from the source to every other node.

Example: adjacency list of a graph, src = 0 → Output: dist[] = shortest hop-count to each node

```
vector<int> bfsShortestPath(int src, vector<vector<int>>& adj, int n) {
    vector<int> dist(n, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
        }
    }
    return dist;
}
```

Time: $O(V+E)$. **Space:** $O(V)$.

Practice: [Word Ladder](#) · [Rotting Oranges](#)

Dijkstra's Algorithm (shortest path, non-negative weights)

What it is: Like BFS, but for weighted edges: always expand outward from whichever unvisited node currently has the smallest known distance (tracked with a min-heap), and update its neighbours' distances if a shorter path was just found.

Recognize it by: "shortest/cheapest path" with weighted edges, all weights non-negative (e.g. travel time, cost).

Intuition: Greedily pick the unvisited node with smallest known distance (via min-heap), relax its neighbors.

Problem: Given a weighted graph with non-negative edge weights, a source node, and a target node, find the shortest travel time (or determine it's unreachable).

Example: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, source k = 2 → Output: 2 (time for the signal to reach all nodes)

Dry Run:

```
pop (dist=0, node=2)
  relax edge 2->1 (w=1): dist[1] inf -> 1
  relax edge 2->3 (w=1): dist[3] inf -> 1
pop (dist=1, node=1)
pop (dist=1, node=3)
  relax edge 3->4 (w=1): dist[4] inf -> 2
pop (dist=2, node=4)
Answer: dist[] from node 2 = [1, 0, 1, 2]
```

```
vector<long long> dijkstra(int src, vector<vector<pair<int,int>>>& adj, int n) {
    vector<long long> dist(n, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // stale entry
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Time: $O((V+E) \log V)$. **Space:** $O(V+E)$.

Practice: [Network Delay Time](#) · [Path with Maximum Probability](#)

Union-Find (Disjoint Set Union) with path compression + union by rank

What it is: A structure for tracking "which group is each item in" and quickly merging two groups together. "Path compression" flattens the lookup chain as you go so future lookups are almost instant.

Recognize it by: "connected components", "are these two nodes connected", "will adding this edge create a cycle", "number of provinces/groups".

Intuition: Efficiently track connected components; near- $O(1)$ union/find amortized.

Problem: Given n cities and a list of direct connections between some of them, find the number of connected 'provinces' (groups of cities reachable from each other).

Example: isConnected = [[1,1,0],[1,1,0],[0,0,1]] → Output: 2 provinces

Dry Run:

```

union(0,1): merge root 0 into root 1 -> parent=[1, 1, 2, 3, 4, 5]
union(1,2): merge root 1 into root 2 -> parent=[1, 2, 2, 3, 4, 5]
union(2,3): merge root 2 into root 3 -> parent=[1, 2, 3, 3, 4, 5]
union(0,3): already same group (root 3) -> cycle would form!
    edge (0,3) is redundant – it closes a cycle

class DSU {
    vector<int> parent, rank_;
public:
    DSU(int n) : parent(n), rank_(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    bool unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false; // already connected (cycle if edge!)
        if (rank_[px] < rank_[py]) swap(px, py);
        parent[py] = px;
        if (rank_[px] == rank_[py]) rank_[px]++;
        return true;
    }
};

```

Time: $O(a(n)) \approx O(1)$ amortized per operation. **Space:** $O(n)$.

Practice: [Number of Provinces](#) · [Redundant Connection](#) · [Accounts Merge](#)

Kruskal's MST (uses DSU)

What it is: To connect all nodes as cheaply as possible: sort every edge by weight, cheapest first, and greedily add an edge only if it doesn't create a cycle (checked instantly using Union-Find).

Recognize it by: "minimum cost to connect all points/cities", "minimum spanning tree" — connect everything as cheaply as possible with no cycles.

Problem: Given a set of points on a plane, connect all of them using the minimum possible total edge cost (a Minimum Spanning Tree), where the cost between two points is their distance.

Example: points = $[[0,0],[2,2],[3,10],[5,2],[7,0]] \rightarrow$ Output: minimum total cost = 20

```

long long kruskalMST(int n, vector<array<int,3>>& edges) { // {weight, u, v}
    sort(edges.begin(), edges.end());
    DSU dsu(n);
    long long total = 0;
    for (auto& [w, u, v] : edges)
        if (dsu.unite(u, v)) total += w;
    return total;
}

```

Time: $O(E \log E)$. **Space:** $O(V+E)$.

Practice: [Min Cost to Connect All Points](#)

Topological Sort (Kahn's Algorithm — BFS based)

What it is: Find a valid "order to do things in" when some tasks depend on others finishing first: repeatedly pick off any task that currently has no remaining unfinished dependencies (in-degree 0), then update the tasks that depended on it.

Recognize it by: "course schedule", "build order", "task dependencies" — anything with a "must happen before" ordering constraint.

Intuition: Repeatedly remove nodes with in-degree 0; valid only for DAGs.

Problem: Given numCourses and a list of prerequisite pairs [a, b] (b must be finished before a), determine a valid order to take all courses, or detect that it's impossible.

Example: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]] → Output: [0,1,2,3] (one valid order)

```
vector<int> topoSort(int n, vector<vector<int>>& adj) {
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; u++)
        for (int v : adj[u]) indeg[v]++;
    queue<int> q;
    for (int i = 0; i < n; i++) if (indeg[i] == 0) q.push(i);
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order.size() == n ? order : vector<int>{}; // empty = cycle detected
}
```

Time: $O(V+E)$. **Space:** $O(V)$.

Practice: [Course Schedule](#) · [Course Schedule II](#) · [Alien Dictionary \(Premium\)](#)

Cycle Detection

What it is: Figure out if a graph loops back on itself. For undirected graphs, Union-Find catches a cycle the moment an edge connects two nodes already in the same group. For directed graphs, DFS tracks nodes "currently being explored" — revisiting one of those means you've found a loop.

Recognize it by: "detect a cycle in a graph", "is this graph a valid tree", "find the redundant edge".

- **Undirected graph:** DSU (unite returns false → cycle) or DFS with parent tracking.
- **Directed graph:** DFS with 3 colors (white/gray/black) — a back-edge to a gray node means cycle; or check topo sort produces fewer than n nodes.

Problem: Given a list of edges forming a graph that was originally a tree but has one extra edge added, find that redundant edge which creates a cycle.

Example: edges = [[1,2],[1,3],[2,3]] → Output: [2,3] is the redundant edge

```
bool hasCycleDirected(int u, vector<vector<int>>& adj, vector<int>& state) {
    // state: 0=unvisited, 1=in progress, 2=done
    state[u] = 1;
    for (int v : adj[u]) {
        if (state[v] == 1) return true; // back edge -> cycle
        if (state[v] == 0 && hasCycleDirected(v, adj, state)) return true;
    }
    state[u] = 2;
    return false;
}
```

Time: $O(V+E)$. **Space:** $O(V)$.

Practice: [Redundant Connection](#) · [Course Schedule](#)

16. Dynamic Programming

Intuition: DP = recursion + memoization (avoid recomputing overlapping subproblems), OR build up bottom-up using a table. Recognize DP when: (1) the problem asks for optimal value / count of ways, and (2) it has **overlapping subproblems** + **optimal substructure** (solution built from solutions to smaller versions of itself).

General approach:

1. Define state: what parameters uniquely describe a subproblem? `dp[i]`, `dp[i][j]`, etc.
2. Define recurrence: how does `dp[i]` relate to smaller states?
3. Base case.
4. Decide direction: top-down (recursion + memo) or bottom-up (iterative table).

16a. 1D DP — Fibonacci / Climbing Stairs

What it is: The answer for step n only depends on the answers for a couple of steps before it. Instead of recomputing those smaller answers every time (which gets exponentially slow), store them once and reuse them.

Recognize it by: the answer for n clearly depends only on the answers for a couple of smaller values just before it ($n-1$, $n-2$, ...).

Problem: You're climbing a staircase of n steps; each move you can climb 1 or 2 steps. Count how many distinct ways there are to reach the top.

Example: $n = 3 \rightarrow$ Output: 3 (1+1+1, 1+2, 2+1)

Dry Run:

```
dp[1]=1, dp[2]=2
dp[3] = dp[2](2) + dp[1](1) = 3
dp[4] = dp[3](3) + dp[2](2) = 5
dp[5] = dp[4](5) + dp[3](3) = 8
Answer: dp[5] = 8
```

```
// Top-down memoization
int memo[10001];
int climbStairs(int n) {
    if (n <= 2) return n;
    if (memo[n]) return memo[n];
    return memo[n] = climbStairs(n-1) + climbStairs(n-2);
}

// Bottom-up, O(1) space
int climbStairsBU(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) { int c = a + b; a = b; b = c; }
    return b;
}
```

Time: $O(n)$. **Space:** $O(n)$ memo version, $O(1)$ optimized.

Practice: [Climbing Stairs](#) · [House Robber](#)

16b. 0/1 Knapsack

What it is: For every item, you make a binary choice: take it or skip it — you can't take a fraction of it or take it twice. Build a table where `dp[i][w]` answers "what's the best value using the first i items within weight limit w ".

Recognize it by: "each item can be used at most once", "partition into two equal subsets", "can we make exactly this sum" with a hard include/exclude choice per item.

Intuition: For each item, decide include or exclude. $dp[i][w]$ = best value using first i items with capacity w .

Problem: Given an array of positive integers, determine whether it can be split into two subsets whose sums are equal.

Example: $nums = [1,5,11,5] \rightarrow$ Output: true ($\{1,5,5\}$ and $\{11\}$ both sum to 11)

Dry Run:

total=22, need subset summing to target=11

considering item 1:

dp[1] becomes True (via $dp[0] + \text{item } 1$)

considering item 5:

dp[6] becomes True (via $dp[1] + \text{item } 5$)

dp[5] becomes True (via $dp[0] + \text{item } 5$)

considering item 11:

dp[11] becomes True (via $dp[0] + \text{item } 11$)

considering item 5:

dp[10] becomes True (via $dp[5] + \text{item } 5$)

dp[11] = True $\rightarrow$ Answer: True

```
int knapsack01(vector<int>& wt, vector<int>& val, int W) {
    int n = wt.size();
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i-1][w]; // exclude item i
            if (wt[i-1] <= w)
                dp[i][w] = max(dp[i][w], dp[i-1][w - wt[i-1]] + val[i-1]); // include
        }
    }
    return dp[n][W];
}

// Space-optimized: 1D array, iterate w from high to low
int knapsack01optimized(vector<int>& wt, vector<int>& val, int W) {
    vector<int> dp(W+1, 0);
    for (int i = 0; i < wt.size(); i++)
        for (int w = W; w >= wt[i]; w--)
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
}
```

Time: $O(n \cdot W)$. **Space:** $O(n \cdot W)$ or $O(W)$ optimized.

Practice: [Partition Equal Subset Sum](#) · [Target Sum](#)

16c. Unbounded Knapsack — Coin Change (min coins)

What it is: Similar to 0/1 knapsack, but this time each item (coin) can be reused an unlimited number of times — so the table doesn't need to track "which items used", just "what's the best answer for this remaining amount".

Recognize it by: "minimum/number of ways to make an amount" where each item/coin CAN be reused unlimited times.

Problem: Given a set of coin denominations and a target amount, find the fewest coins needed to make that amount (or -1 if it can't be made).

Example: $coins = [1,2,5]$, $amount = 11 \rightarrow$ Output: 3 ($5+5+1$)

Dry Run:

```
dp[1] = 1
dp[2] = 1
dp[5] = 1
dp[6] = 2
dp[10] = 2
dp[11] = 3
Answer: dp[11] = 3
```

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins)
            if (c <= a && dp[a-c] != INT_MAX)
                dp[a] = min(dp[a], dp[a-c] + 1);
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
```

Time: $O(\text{amount} \times \text{coins.size}())$. **Space:** $O(\text{amount})$.

Practice: [Coin Change](#) · [Coin Change II](#)

16d. Longest Common Subsequence (2D DP on strings)

What it is: Build a 2D grid where $dp[i][j]$ represents "the answer using the first i characters of string A and the first j characters of string B" — if the current characters match, extend a previous diagonal answer; otherwise take the best of skipping a character from either string.

Recognize it by: comparing TWO strings/sequences and the answer depends on characters matching or not — LCS, edit distance, shortest common supersequence all share this table shape.

Problem: Given two strings, find the length of their longest common subsequence (characters in the same relative order, not necessarily contiguous).

Example: "abcde" and "ace" → Output: 3 ("ace")

Dry Run:

DP table (rows=a, cols=b):

	a	c	e
0	0	0	0
a	0	1	1
b	0	1	1
c	0	1	2
d	0	1	2
e	0	1	2

Answer: $dp[5][3] = 3$

```
int longestCommonSubsequence(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1]) ? dp[i-1][j-1] + 1
                                           : max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}
```

Time: $O(n \cdot m)$. **Space:** $O(n \cdot m)$ (can optimize to $O(\min(n, m))$).

Practice: [Longest Common Subsequence](#) · [Edit Distance](#) · [Shortest Common Supersequence](#) **Same table powers:** edit distance, longest common substring, shortest common supersequence.

16e. Longest Increasing Subsequence

What it is: $dp[i]$ means "the longest increasing run of numbers that ENDS exactly at index i ". To fill it in, look back at every earlier index with a smaller value and see which one gives the longest chain so far.

Recognize it by: "longest increasing/decreasing subsequence" — order must be preserved, but elements don't have to be adjacent.

Intuition ($O(n^2)$): $dp[i] = \text{LIS ending at index } i = 1 + \max(dp[j])$ for all $j < i$ with $a[j] < a[i]$.

Problem: Given an integer array, find the length of the longest strictly increasing subsequence.

Example: `nums = [10,9,2,5,3,7,101,18]` → Output: 4 (`[2,3,7,101]` or `[2,3,7,18]`)

Dry Run:

```
x=10: extends tails -> [10]
x=9: replaces tails[0]=10 -> [9]
x=2: replaces tails[0]=9 -> [2]
x=5: extends tails -> [2, 5]
x=3: replaces tails[1]=5 -> [2, 3]
x=7: extends tails -> [2, 3, 7]
x=101: extends tails -> [2, 3, 7, 101]
x=18: replaces tails[3]=101 -> [2, 3, 7, 18]
Answer: length = 4
```

```
int lengthOfLIS(vector<int>& a) {
    int n = a.size();
    vector<int> dp(n, 1);
    int best = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
    for (int x : dp) best = max(best, x);
    return best;
}
```

Time: $O(n^2)$. **Space:** $O(n)$.

Practice: [Longest Increasing Subsequence](#) · [Russian Doll Envelopes](#)

Intuition ($O(n \log n)$): Maintain an array `tails` where `tails[k]` = smallest possible tail of an increasing subsequence of length $k+1$. Binary search to place each element.

```
int lengthOfLIS_fast(vector<int>& a) {
    vector<int> tails;
    for (int x : a) {
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if (it == tails.end()) tails.push_back(x);
        else *it = x;
    }
    return tails.size();
}
```

Time: $O(n \log n)$. **Space:** $O(n)$.

16f. Grid DP — Unique Paths / Min Path Sum

What it is: Moving only right or down through a grid, the best way to reach any cell depends only on the best way to reach the cell above it and the cell to its left — so fill the grid top-left to bottom-right, reusing answers as you go.

Recognize it by: moving through a 2D grid only right/down (or similar restricted moves), optimizing a sum or counting paths.

Problem: Given an $m \times n$ grid of non-negative numbers, find a path from the top-left to the bottom-right corner (moving only right or down) that minimizes the sum of numbers along the path.

Example: grid = [[1,3,1],[1,5,1],[4,2,1]] → Output: 7 (path 1→3→1→1→1)

Dry Run:

DP table (min cost to reach each cell):

```
1  4  5
2  7  6
6  8  7
```

Answer: dp[2][2] = 7

```
int minPathSum(vector<vector<int>>& grid) {
    int n = grid.size(), m = grid[0].size();
    vector<vector<int>> dp(n, vector<int>(m));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            if (i == 0 && j == 0) dp[i][j] = grid[i][j];
            else if (i == 0) dp[i][j] = dp[i][j-1] + grid[i][j];
            else if (j == 0) dp[i][j] = dp[i-1][j] + grid[i][j];
            else dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j];
        }
    return dp[n-1][m-1];
}
```

Time: $O(n \cdot m)$. **Space:** $O(n \cdot m)$, optimizable to $O(m)$.

Practice: [Unique Paths](#) · [Unique Paths II](#) · [Minimum Path Sum](#)

16g. DP on Trees

What it is: Same "combine children's answers" idea as tree height/diameter, but now used to solve an optimization problem — each node computes its best answer using the (already computed) best answers of its children.

Recognize it by: an optimization question (max/min sum, longest path, etc.) on a tree, where each node's answer depends on combining its children's answers.

Intuition: Compute a value bottom-up via post-order traversal; each node combines results from its children.

Problem: Given the root of a binary tree, find the maximum sum of any path between two nodes (the path doesn't need to pass through the root).

Example: tree = [1,2,3] → Output: 6 (path 2→1→3)

```
// Max path sum in a binary tree (classic tree DP)
int maxPathSum(TreeNode* root, int& best) {
    if (!root) return 0;
    int left = max(0, maxPathSum(root->left, best)); // ignore negative branches
    int right = max(0, maxPathSum(root->right, best));
    best = max(best, root->val + left + right); // path through this node
    return root->val + max(left, right); // best path if extending upward
}
```

Time: $O(n)$. **Space:** $O(h)$.

Practice: [Binary Tree Maximum Path Sum](#) · [House Robber III](#)

16h. Bitmask DP (Traveling Salesman style)

What it is: When n is small ($\leq \sim 20$), represent "which subset of items have been used so far" as a single integer where each bit means one item — this lets the DP state be just (bitmask, current position) instead of needing a whole separate array per subset.

Recognize it by: n is small ($\leq \sim 20$) and the state needs to remember "which subset of items has been used/visited so far".

Intuition: When n is small ($\leq \sim 20$) and state includes "which subset of items used", encode subset as a bitmask.

Problem: Given n cities and the distance between every pair, find the minimum-cost route that visits every city exactly once and returns to the starting city.

Example: 4 cities with a given distance matrix → Output: the minimum total tour cost

Dry Run:

```
state(mask=0b111, pos=2) -> best cost from here = 50
state(mask=0b1011, pos=3) -> best cost from here = 45
state(mask=0b11, pos=1) -> best cost from here = 70
state(mask=0b111, pos=1) -> best cost from here = 45
state(mask=0b1101, pos=3) -> best cost from here = 35
state(mask=0b101, pos=2) -> best cost from here = 65
... (10 states total explored)
Answer: minimum tour cost = 80
```

```
int n;
vector<vector<int>>> dist;
vector<vector<int>>> dp; // dp[mask][i] = min cost visiting set `mask`, ending at i
int tsp(int mask, int pos) {
    if (mask == (1 << n) - 1) return dist[pos][0]; // return to start
    if (dp[mask][pos] != -1) return dp[mask][pos];
    int ans = INT_MAX;
    for (int city = 0; city < n; city++) {
        if (!(mask & (1 << city))) {
            int newAns = dist[pos][city] + tsp(mask | (1 << city), city);
            ans = min(ans, newAns);
        }
    }
    return dp[mask][pos] = ans;
}
```

Time: $O(n^2 \cdot 2^n)$. **Space:** $O(n \cdot 2^n)$.

Practice: [Shortest Path Visiting All Nodes](#) · [Partition to K Equal Sum Subsets](#)

17. Greedy Algorithms

Intuition: Make the locally optimal choice at each step, hoping it leads to a global optimum. Works only when the problem has the **greedy-choice property** — prove it (or trust it's a known greedy problem) before using it; unlike DP, greedy doesn't explore alternatives.

Activity Selection / Interval Scheduling (max non-overlapping intervals)

What it is: Sort all the intervals by their END time, then greedily keep picking whichever remaining interval finishes soonest — finishing early always leaves the most room for whatever comes after.

Recognize it by: "maximum non-overlapping intervals", "minimum intervals to remove so none overlap", "maximum meetings you can attend".

Intuition: Always pick the interval that finishes earliest — it leaves the most room for future intervals.

Problem: Given a list of intervals, find the maximum number of them that can be kept without any two overlapping (equivalently, the minimum number to remove).

Example: intervals = [[1,2],[2,3],[3,4],[1,3]] → Output: keep 3 (remove just [1,3])

```
int maxNonOverlapping(vector<pair<int,int>>& intervals) { // {start, end}
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) {
        return a.second < b.second; // sort by end time
    });
    int count = 0, lastEnd = INT_MIN;
    for (auto& [s, e] : intervals) {
        if (s >= lastEnd) { count++; lastEnd = e; }
    }
    return count;
}
```

Time: $O(n \log n)$. **Space:** $O(1)$ extra (excluding sort).

Practice: [Non-overlapping Intervals](#)

Merge Intervals

What it is: Sort intervals by start time, then walk through them once: if the next interval overlaps with the one you're currently building, stretch it to cover both; otherwise, close it off and start a new one.

Recognize it by: "merge overlapping intervals", "insert an interval" into an existing sorted list of intervals.

Problem: Given a list of intervals, merge all intervals that overlap and return the resulting non-overlapping set.

Example: intervals = [[1,3],[2,6],[8,10],[15,18]] → Output: [[1,6],[8,10],[15,18]]

```
vector<vector<int>> mergeIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> res;
    for (auto& iv : intervals) {
        if (!res.empty() && iv[0] <= res.back()[1])
            res.back()[1] = max(res.back()[1], iv[1]);
        else
            res.push_back(iv);
    }
    return res;
}
```

Time: $O(n \log n)$. **Space:** $O(n)$.

Practice: [Merge Intervals](#) · [Insert Interval](#)

Fractional Knapsack

What it is: Unlike 0/1 knapsack, here you're ALLOWED to take a fraction of an item. So greedily grab items with the best value-per-weight ratio first, and only take a partial amount of the last item that doesn't fully fit.

Recognize it by: items CAN be split/taken partially (unlike 0/1 knapsack) and you're maximizing value under a weight/capacity limit.

Problem: Given items each with a value and a weight, and a knapsack of limited capacity, maximize total value where items CAN be taken fractionally.

Example: items(value,weight) = [(60,10),(100,20),(120,30)], W = 50 → Output: 240

```
double fractionalKnapsack(vector<pair<int,int>>& items, int W) { // {value, weight}
    sort(items.begin(), items.end(), [](auto& a, auto& b) {
        return (double)a.first / a.second > (double)b.first / b.second; // value/weight desc
    });
    double totalValue = 0;
    for (auto& [val, wt] : items) {
        if (W >= wt) { totalValue += val; W -= wt; }
        else { totalValue += val * ((double)W / wt); break; }
    }
    return totalValue;
}
```

Time: $O(n \log n)$. **Space:** $O(1)$.

Practice: [Maximum Units on a Truck](#)

Jump Game (greedy reachability)

What it is: Track the furthest index you could possibly reach so far as you scan left to right. If at some point your current position is already beyond that furthest-reachable point, you're stuck and can never get further.

Recognize it by: "can you reach the last index", "minimum jumps to reach the end" — each position tells you how far you're allowed to jump.

Problem: Given an array where each element is the maximum jump length from that position, determine whether you can reach the last index starting from the first.

Example: nums = [2,3,1,1,4] → Output: true. nums = [3,2,1,0,4] → Output: false

```
bool canJump(vector<int>& nums) {
    int reach = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (i > reach) return false;
        reach = max(reach, i + nums[i]);
    }
    return true;
}
```

Time: $O(n)$. **Space:** $O(1)$.

Practice: [Jump Game](#) · [Jump Game II](#) · [Gas Station](#)

18. Bit Manipulation

Trick	Code	Use	
Check bit i	<code>(n >> i) & 1</code>	test if bit set	
Set bit i	<code>n (1 << i)</code>	turn bit on	
Clear bit i	<code>n & ~(1 << i)</code>	turn bit off	
Toggle bit i	<code>n ^ (1 << i)</code>	flip bit	
Remove lowest set bit	<code>n & (n - 1)</code>	count set bits, power of 2 check	

Trick	Code	Use	
Isolate lowest set bit	<code>n & (-n)</code>	Fenwick tree indexing	
Check power of 2	<code>n > 0 && (n & (n-1)) == 0</code>	—	

Count set bits (Brian Kernighan's algorithm)

What it is: The trick `n & (n - 1)` always clears the LOWEST set bit of `n` in one operation — so repeating it until `n` becomes 0 counts exactly how many 1-bits `n` had, without checking every single bit position one by one.

Recognize it by: "count the number of 1 bits", "Hamming weight", or any problem repeatedly checking/clearing individual bits.

Problem: Given a non-negative integer, count the number of 1-bits in its binary representation (Hamming weight).

Example: `n = 11` (binary 1011) → Output: 3

```
int countSetBits(int n) {
    int count = 0;
    while (n) { n &= (n - 1); count++; }
    return count;
}
```

Time: $O(\text{number of set bits})$. **Space:** $O(1)$.

Practice: [Number of 1 Bits](#) · [Counting Bits](#)

Single Number (XOR trick)

What it is: XOR-ing a number with itself gives 0, and XOR-ing with 0 changes nothing — so if every number in an array appears in a pair except one, XOR-ing everything together cancels out all the pairs and leaves only the lonely number.

Recognize it by: "every element appears twice except one" (or similar parity-based uniqueness) — a strong hint to try XOR.

Intuition: `a ^ a = 0`, `a ^ 0 = a` — XOR-ing all elements cancels out pairs, leaving the unique one.

Problem: Given a non-empty array where every element appears twice except for one, find that single element.

Example: `nums = [4,1,2,1,2]` → Output: 4

```
int singleNumber(vector<int>& nums) {
    int result = 0;
    for (int x : nums) result ^= x;
    return result;
}
```

Time: $O(n)$. **Space:** $O(1)$.

Practice: [Single Number](#) · [Single Number II](#) · [Single Number III](#)

Generate all subsets using bitmasking

What it is: Every number from 0 to $2^n - 1$, written in binary, is a unique pattern of "included / not included" for `n` items — so looping through all those numbers and reading their bits generates every possible subset.

Recognize it by: `n` is small ($\leq \sim 20$) and you need every subset — bitmasking is a compact non-recursive alternative to backtracking subsets.

Problem: Given an array of distinct integers, generate every possible subset (the power set) by treating each number from 0 to $2^n - 1$ as an inclusion/exclusion mask.

Example: nums = [1,2,3] → Output: all 8 subsets, e.g. mask=101 → {1,3}

```
vector<vector<int>> subsetsBitmask(vector<int>& nums) {
    int n = nums.size();
    vector<vector<int>> res;
    for (int mask = 0; mask < (1 << n); mask++) {
        vector<int> subset;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) subset.push_back(nums[i]);
        res.push_back(subset);
    }
    return res;
}
```

Time: $O(2^n \cdot n)$. **Space:** $O(2^n \cdot n)$ output.

Practice: [Subsets](#) · [Power of Two](#)

19. Math for DSA

GCD / LCM

What it is: The Greatest Common Divisor of two numbers can be found by repeatedly replacing the bigger number with the remainder of dividing it by the smaller one (Euclid's algorithm) — it shrinks fast, so this converges in very few steps.

Recognize it by: reducing fractions, finding a common cycle length, or any "largest number that evenly divides both" question.

Problem: Given two positive integers, find their Greatest Common Divisor and Least Common Multiple.

Example: a = 12, b = 18 → Output: gcd = 6, lcm = 36

```
long long gcd(long long a, long long b) { return b == 0 ? a : gcd(b, a % b); }
long long lcm(long long a, long long b) { return a / gcd(a, b) * b; }
```

Time: $O(\log(\min(a,b)))$. **Space:** $O(\log(\min(a,b)))$ recursion (or $O(1)$ iterative).

Practice: [Greatest Common Divisor of Strings](#)

Sieve of Eratosthenes (all primes up to n)

What it is: Starting from 2, cross out every multiple of each prime you find — whatever numbers are never crossed out by the time you finish are exactly the primes. Much faster than checking each number individually for primality.

Recognize it by: you need ALL primes up to some n, not just a single primality check — precompute once, answer many queries fast.

Problem: Given an integer n, find all prime numbers less than or equal to n.

Example: n = 10 → Output: [2, 3, 5, 7]

```
vector<bool> sieve(int n) {
    vector<bool> isPrime(n + 1, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; (long long)i * i <= n; i++)
```

```

        if (isPrime[i])
            for (int j = i * i; j <= n; j += i)
                isPrime[j] = false;
    return isPrime;
}

```

Time: $O(n \log \log n)$. **Space:** $O(n)$.

Practice: [Count Primes](#)

Fast Exponentiation (modular power)

What it is: To compute base^{exp} , repeatedly square the base and halve the exponent (using the binary representation of the exponent) instead of multiplying base by itself exp times — turns a linear-time operation into a logarithmic one.

Recognize it by: computing a huge power modulo something ("mod $1e9+7$ "), especially with an exponent too large to loop through directly.

Problem: Compute $(\text{base}^{\text{exp}}) \bmod m$ efficiently, even when exp is extremely large.

Example: base = 2, exp = 10, mod = 1_000_000_007 → Output: 1024

```

long long power(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) result = (result * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}

```

Time: $O(\log \text{exp})$. **Space:** $O(1)$.

Practice: [Pow\(x, n\)](#) · [Super Pow](#)

Factorial / nCr with modular inverse (for large n)

What it is: Since normal division doesn't work cleanly under a modulus, "dividing" by a number modulo a prime is done by multiplying by its modular inverse instead (computed via fast exponentiation, using Fermat's Little Theorem).

Recognize it by: "number of ways to choose", combinatorics questions with results requested modulo a large prime.

Problem: Compute nCr (n choose r) modulo a large prime, for n large enough that factorials overflow normal integers.

Example: n = 5, r = 2, MOD = $1e9+7$ → Output: 10 ($5C2 = 10$)

```

const int MOD = 1e9 + 7;
vector<long long> fact;
void precomputeFactorials(int n) {
    fact.assign(n + 1, 1);
    for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * i % MOD;
}
long long modInverse(long long a) { return power(a, MOD - 2, MOD); } // Fermat's little theorem
long long nCr(int n, int r) {
    if (r > n || r < 0) return 0;
    return fact[n] * modInverse(fact[r]) % MOD * modInverse(fact[n-r]) % MOD;
}

```

}

Time: $O(n)$ precompute, $O(\log \text{MOD})$ per query. **Space:** $O(n)$.

Practice: [Unique Paths](#)

20. How to Practice (Roadmap)

1. **Arrays/Strings/Two Pointers/Sliding Window** (1–2 weeks) — build pattern recognition first, this is 30% of interview questions.
2. **Sorting + Binary Search** (3–5 days) — especially "binary search on answer".
3. **Recursion/Backtracking** (1 week) — draw the recursion tree for every problem until it's automatic.
4. **Linked List, Stack, Queue** (1 week) — mostly pointer manipulation, low conceptual overhead once patterns click.
5. **Trees + BST** (1–2 weeks) — master traversals cold, then move to tree DP and LCA-family problems.
6. **Heaps + Tries** (3–5 days).
7. **Graphs** (2 weeks) — BFS/DFS first, then Dijkstra/Union-Find/MST/Topo sort.
8. **Dynamic Programming** (2–3 weeks, the hardest, highest ROI topic) — go pattern by pattern: 1D → knapsack family → LCS family → grid DP → tree DP → bitmask DP.
9. **Greedy + Bit Manipulation + Math** (1 week) — smaller topics but frequently used as sub-routines.

Habits that matter more than volume:

- For every problem: state the **pattern** it belongs to before coding.
- Always state **time and space complexity** out loud, even in practice.
- Redo problems you struggled with after 3–4 days (spaced repetition) rather than only doing new ones.
- Once comfortable, time yourself: 25–35 minutes per medium problem is the interview bar.

This handbook covers the core pattern set used across almost all placement-level DSA rounds (product-based and service-based companies). For company-specific practice, layer in their most frequently asked problems on top of these patterns — the underlying technique will almost always be one from this document.